REPORT

PHASE – 1

Screenshot 1A

```
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$
```

Screenshot 1B

```
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$
```

PHASE – 2

Screenshot 2A

```
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B

```
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ find .pes/objects -type f
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/b2/fd0fd6d27cb16957f7bde2a2f6ffc5f03b190da82df38e9aaf9061138f9a0d
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/91/4b8f23b2049bacf1f3b77183fcceb18856ac892fc9ef1209d3ed4dd213c70f
.pes/objects/32/384adec4644a54803d997bd6c8c35bbb21c07f73598d9224a5dfd4e1e1449a
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ for f in $(find .pes/objects -type f); do
    echo "---- $f ----"
    xxd $f | head -10
done
---- .pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d ----
00000000: 626c 6f62 2031 3200 6865 6c6c 6f20 776f  blob 12.hello wo
00000010: 726c 640a                                rld.
---- .pes/objects/b2/fd0fd6d27cb16957f7bde2a2f6ffc5f03b190da82df38e9aaf9061138f9a0d ----
00000000: 636f 6d6d 6974 2032 3438 0074 7265 6520  commit 248.tree
00000010: 3665 6631 3962 3431 3232 3563 3533 3639  6ef19b41225c5369
00000020: 6631 6331 3034 6434 3564 3864 3835 6566  f1c104d45d8d85ef
00000030: 6139 6230 3537 6235 3362 3134 6234 6239  a9b057b53b14b4b9
00000040: 6239 3339 6464 3734 6465 6363 3533 3231  b939dd74decc5321
00000050: 0a70 6172 656e 7420 3931 3462 3866 3233  .parent 914b8f23
00000060: 6232 3034 3962 6163 6631 6633 6237 3731  b2049bacf1f3b771
00000070: 3833 6663 6365 6231 3838 3536 6163 3839  83fcceb18856ac89
00000080: 3266 6339 6566 3132 3039 6433 6564 3464  2fc9ef1209d3ed4d
00000090: 6432 3133 6337 3066 0a61 7574 686f 7220  d213c70f.author
---- .pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321 ----
00000000: 7472 6565 2030 00                        tree 0.
---- .pes/objects/91/4b8f23b2049bacf1f3b77183fcceb18856ac892fc9ef1209d3ed4dd213c70f ----
00000000: 636f 6d6d 6974 2031 3733 0074 7265 6520  commit 173.tree
00000010: 3665 6631 3962 3431 3232 3563 3533 3639  6ef19b41225c5369
00000020: 6631 6331 3034 6434 3564 3864 3835 6566  f1c104d45d8d85ef
00000030: 6139 6230 3537 6235 3362 3134 6234 6239  a9b057b53b14b4b9
00000040: 6239 3339 6464 3734 6465 6363 3533 3231  b939dd74decc5321
00000050: 0a61 7574 686f 7220 596f 7572 204e 616d  .author Your Nam
00000060: 6520 3c50 4553 5855 4732 3443 5336 3133  e <PESXUG24CS613
00000070: 3e20 3137 3736 3433 3730 3139 0a63 6f6d  > 1776437019.com
00000080: 6d69 7474 6572 2059 6f75 7220 4e61 6d65  mitter Your Name
00000090: 203c 5045 5358 5547 3234 4353 3631 333e   <PESXUG24CS613>
---- .pes/objects/32/384adec4644a54803d997bd6c8c35bbb21c07f73598d9224a5dfd4e1e1449a ----
00000000: 626c 6f62 2031 3000 7465 7374 2064 6174  blob 10.test dat
00000010: 610a                                     a.
---- .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 ----
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$
```

PHASE – 3

Screenshot 3A

```
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ ./pes status
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  test_tree
  untracked:  commit.h
  untracked:  tree.h
  untracked:  .gitignore
  untracked:  index.h
  untracked:  test_sequence.sh
  untracked:  file2.txt
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  test.txt
  untracked:  pes.c
  untracked:  test_objects.c
  untracked:  index.c
  untracked:  pes.h
  untracked:  Makefile
  untracked:  file1.txt
  untracked:  README.md
```

Screenshot 3B

```
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ ./pes add file1.txt file2.txt
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776440702 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776440704 6 file2.txt
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$
```

PHASE- 4

Screenshot 4A

```
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ ./pes log
commit 929c82de60dbd59ff087d427ea532d6eae25b74aba0d8b86a18024d81d38bf24
Author: Your Name <PESXUG24CS613>
Date:   1776440958

    Add farewell

commit 36a84d04a53e81ea124ed719eacc878e655f4a9ca6fe9911610447442cba9fb0
Author: Your Name <PESXUG24CS613>
Date:   1776440941

    Add world

commit 2349c918f09c9a6d111a5913fc688e7e8095f66b2f638fad9bcc76c252896bb0
Author: Your Name <PESXUG24CS613>
Date:   1776440923

    Initial commit

yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$
```

Screenshot 4B

```
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/23/49c918f09c9a6d111a5913fc688e7e8095f66b2f638fad9bcc76c252896bb0
.pes/objects/36/a84d04a53e81ea124ed719eacc878e655f4a9ca6fe9911610447442cba9fb0
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/92/9c82de60dbd59ff087d427ea532d6eae25b74aba0d8b86a18024d81d38bf24
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ S
```

Screenshot 4C

```
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$ cat .pes/refs/heads/main
929c82de60dbd59ff087d427ea532d6eae25b74aba0d8b86a18024d81d38bf24
yashita@yashita-VirtualBox:~/PES2UG24CS613-pes-vcs$
```

PHASE – 5 & 6

Q5.1

To implement pes checkout <branch>, we need to:

- Update .pes/HEAD to point to refs/heads/<branch>
- Read the commit hash from .pes/refs/heads/<branch>
- Load the corresponding tree object

- Update the working directory to match that tree (create/update/delete files accordingly)

The operation is complex because:

- Files may need to be added, removed, or overwritten
- Directory structure must be reconstructed from tree objects
- Conflicts can occur if there are uncommitted changes in the working directory

Q5.2
To detect a "dirty working directory":

- Compare each tracked file in the index with the current working directory using metadata (mtime and size)
- If metadata differs, recompute the file hash and compare with the index hash
- Also compare the target branch's tree with the index

If a file:

- is modified locally AND
- differs between current branch and target branch

→ checkout must be refused to avoid overwriting changes.

Q5.3
In a detached HEAD state:

- HEAD points directly to a commit, not a branch
- New commits are created but are not referenced by any branch

These commits can be lost (garbage collected) if not referenced.
To recover them:

- Create a new branch pointing to that commit
  (e.g., pes branch <name> at that commit)
- Or use logs/history to find the commit hash and restore it

Q6.1
To delete unreachable objects:

1. Start from all branch heads (refs/heads/*)

2. Traverse commits recursively via parent links
3. For each commit, include its tree and all referenced blobs/trees
4. Mark all visited objects as “reachable”
5. Delete any object not in this set

Data structure:

- Use a hash set (for O(1) lookup of visited hashes)

Estimate:

- ~100,000 commits + associated trees/blobs
- Likely several hundred thousand objects visited in total

Q6.2
Running GC during a commit is dangerous due to race conditions:

Example:

- Commit process creates a new object but hasn’t updated HEAD yet
- GC runs and sees the object as “unreachable”
- GC deletes it
- Commit then tries to reference it → corruption

Git avoids this by:

- Running GC only when repository is stable
- Using locks and temporary files
- Ensuring new objects are fully written and referenced before GC runs